Chapter 5. The C++ memory model and operations on atomic types

This chapter covers

- The details of the C++11 memory model
- The atomic types provided by the C++ Standard Library
- The operations that are available on those types
- How those operations can be used to provide synchronization between threads

One of the most important features of the C++11 Standard is something most programmers won't even notice. It's not the new syntax features, nor is it the new library facilities, but the new multithreading-aware memory model. Without the memory model to define exactly how the fundamental building blocks work, none of the facilities I've covered could be relied on to work. Of course, there's a reason that most programmers won't notice: if you use mutexes to protect your data and condition variables or futures to signal events, the details of *why* they work aren't important. It's only when you start trying to get "close to the machine" that the precise details of the memory model matter.

Whatever else it is, C++ is a systems programming language. One of the goals of the Standards Committee is that there shall be no need for a lower-level language than C++. Programmers should be provided with enough flexibility within C++ to do whatever they need without the language getting in the way, allowing them to get "close to the machine" when the need arises. The atomic types and operations allow just that, providing facilities for low-level synchronization operations that will commonly reduce to one or two CPU instructions.

In this chapter, I'll start by covering the basics of the memory model, then move on to the atomic types and operations, and finally cover the various types of synchronization available with the operations on atomic types. This is quite complex: unless you're planning on writing code that uses the atomic operations for synchronization (such as the lock-free data structures in chapter 7), you won't need to know these details.

Let's ease into things with a look at the basics of the memory model.

5.1. Memory model basics

There are two aspects to the memory model: the basic *structural* aspects, which relate to how things are laid out in memory, and then the *concurrency* aspects. The structural aspects are important for concurrency, especially when you're looking at low-level atomic operations, so I'll start with those. In C++, it's all about objects and memory locations.